DESKTOP APPLICATION PENETRATION TESTING

From Architecture to Exploitation — With Real-World Case Studies

Agenda

- Intro & Foundations
- Module 1 — Architecture & Threat Modeling
- Module 2 — Static Analysis & Reverse Engineering
- Module 3 — Dynamic Analysis & Runtime Patching
- Module 4 — IPC, Traffic Interception & Local Storage
- Wrap-up, Quiz & Q&A

The App That Turned On Your Webcam

In 2019, a researcher found that Zoom's Mac app secretly installed a hidden local web server — one that stayed running even after you uninstalled Zoom. Any website you visited could quietly use it to start a video call, webcam included.

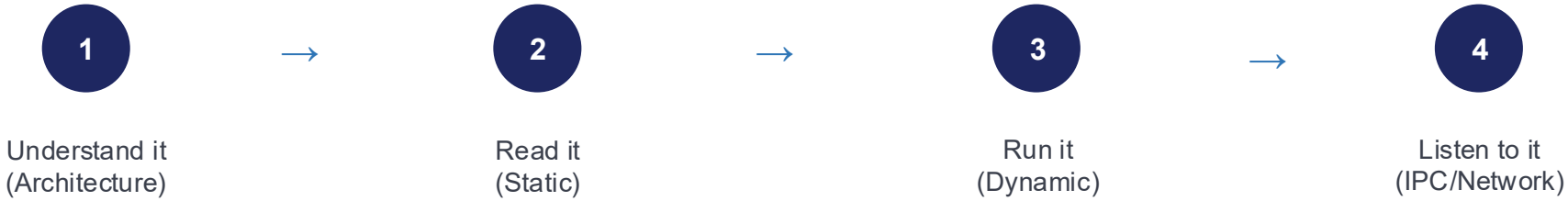
No malware. No phishing. Just the app's own architecture.

What We Mean By “Desktop App”

A program that installs and runs directly on the OS — not in a browser, not on a phone.

Examples you already use: Discord, Slack, VS Code (all Electron), Photoshop, VLC, and internal enterprise “thick clients” for banking, HR, and inventory.

Today's Roadmap



Section 01

Architecture & Threat Modeling

Application types, attack surfaces, security boundaries

Application Types

Type	Example	Notes
Native (C/C++)	VLC, system tools	Fast; memory-corruption bugs live here
Managed (.NET / Java)	Enterprise tools	Runs on CLR/JVM; easy to decompile
Cross-platform (Electron)	Slack, Discord, VS Code	Bundled browser + Node.js; readable JS
Interpreted (PyInstaller)	Python-packaged apps	Often unpacks near original source

What Is “Attack Surface”?

- Every place the app accepts input or touches something outside itself
- Files it opens — documents, imports, plugins
- Network calls — APIs, license checks, auto-update
- Local IPC — other processes talking to it (Section 4)
- GUI input — text fields, drag-and-drop, clipboard
- The installer itself — often the weakest link

Zoom's Hidden Local Web Server

CS

2019 — Researcher Jonathan Leitschuh found Zoom's Mac installer silently ran a local web server so meeting links could auto-join without a browser prompt.

The problem: any website could send a request to that server, because it never checked who was asking. Visiting a malicious page could force a video call to start — webcam included. Apple ultimately force-removed the component from every Mac.

What Is a “Security Boundary”?

- A line separating things with different trust levels — normal user vs. Administrator/SYSTEM, or “my data” vs. “another app's data”
- Enforced by user accounts, file/registry ACLs, process isolation, sandboxing

Most desktop vulnerabilities are really about a security boundary being weaker than it looks. “A low-privilege user tricks a SYSTEM process into acting for them” is the shape of a huge fraction of real findings.

SolarWinds Supply Chain Attack

CS

2020 — Attackers compromised SolarWinds' build server and inserted malicious code into the legitimately signed Orion update, trusted and auto-installed by ~18,000 organizations, including U.S. government agencies.

Not a code bug — a trust-boundary failure. The update mechanism trusted “signed by SolarWinds” as proof of safety, but the signer itself was already compromised.

LastPass 2022 Breach

CS

Attackers compromised a LastPass engineer's personal home computer via a vulnerable third-party media package, installing a keylogger that captured the engineer's master password.

That password gave access to a shared cloud environment holding encryption keys for customer vault backups. LastPass's own app wasn't “broken” — the trust boundary that failed was “we trust this one laptop with keys to everything.”

Threat Modeling With STRIDE

S

Spoofing

Pretending to be someone/something else

T

Tampering

Modifying data or code without authorization

R

Repudiation

Denying an action with no way to prove otherwise

I

Info Disclosure

Exposing data to those not entitled to it

D

DoS

Denying or degrading service to legitimate users

E

Elev. of Privilege

Gaining capabilities without authorization

Section 02

Static Analysis & Reverse Engineering

Decompilation, binary protections, hardcoded secrets

First Contact With a Binary

- file — identifies the file type in seconds
- strings — dumps readable text embedded in the binary
- Hunt for: URLs, file paths, error messages
- ...and very often: hardcoded passwords, API keys, license keys left in plaintext
- Cheap, fast — always do this before anything else

SolarWinds' Hardcoded Password

CS

During the 2020 breach investigation, researchers separately found SolarWinds had protected an internal update server with the password “solarwinds123” — reportedly discoverable via a public GitHub repo years earlier.

Decompilation vs. Disassembly

Term	What It Means	Tools
Disassembly	Machine code → assembly (still low-level)	Ghidra, IDA, radare2
Decompilation	Assembly → pseudo-C, much easier to read	Ghidra, IDA Free
.NET apps	Near-perfect decompilation — structure survives	dnSpy, ILSpy
Java apps	Bytecode decompiles cleanly	jadx, CFR
Electron apps	No decompiling needed — extract app.asar	asar, any text editor

WinRAR's 15 Year Old DLL (CVE-2018-20250)

CS

Check Point Research reverse-engineered UNACEV2.DLL — a 2006-era, closed-source, abandoned third-party library bundled unchanged inside WinRAR for 15 years.

Through static analysis alone, they found it didn't validate file paths inside ACE archives — a malicious archive could extract a file straight into the Windows Startup folder. WinRAR's fix: drop ACE support entirely.

Binary Protections

A

ASLR

Memory addresses shuffle each run — makes exploits unreliable

D

DEP / NX

Marks memory “data only” — can't execute injected data

S

Code Signing

Proves who signed the binary — not that the content is safe

P

Packers

Compress/scramble to slow analysis — unpack themselves at runtime

CCleaner Supply Chain Hack (2017)

CS

Attackers compromised Piriform's build environment and inserted a malware stage into the official, correctly signed CCleaner installer. Over 2 million users downloaded the trojanized version straight from the real vendor.

Reverse engineers found the malware fingerprinted infected machines and delivered a second-stage payload only to a hand-picked list of large tech companies — sifting a huge haystack for high-value targets.

Section 03

Dynamic Analysis & Runtime Patching

Memory modification, API hooking, DLL hijacking

Process & File Monitoring

- Process Monitor / Process Explorer (Windows) — free, essential, no scripting required
- strace / ltrace, dtruss (Linux / macOS equivalents)
- Watch for: file writes outside expected folders, registry access
- Watch for: DLL loads that fail (NAME NOT FOUND) — foreshadows DLL hijacking
- Watch for: unexpected spawned child processes

Print Nightmare’s Discovery Process (CVE-2021-34527)

CS

Researchers investigating the Windows Print Spooler used exactly this kind of monitoring to understand what happened when installing a new printer driver.

They found the RPC interface let a low-privileged — even remote — user install a driver that runs as SYSTEM. It became one of the most talked-about Windows vulnerabilities of 2021, forcing emergency out-of-band patches.

Debuggers & Runtime Patching

- A debugger pauses a running program at a breakpoint, lets you inspect/change memory and registers, then resumes — a remote control for execution
- Windows: x64dbg (beginner-friendly), WinDbg (powerful)
- Linux / macOS: gdb, lldb

EXAMPLE — MEMORY PATCHING

Break right after a license-check function returns true/false, then flip the register manually so the app thinks the license is valid — behavior changed with nothing written to disk.

API Hooking with Frida

CS

Hooking = intercepting a function call at runtime and substituting your own logic before/instead of/after the real one runs.

Frida (cross-platform, scriptable in JavaScript) can hook a CheckLicenseKey function and force it to always return true — no recompiling needed. The same technique is used to bypass certificate pinning: hook the cert-validation function so a proxy's fake certificate is always accepted.

DLL Hijacking

- Windows apps often load DLLs by name only, not full path — Windows searches a defined folder order to find a match
- If an attacker plants a same-named malicious DLL earlier in that search order, the app loads it instead — often at the app's own privilege level
- Finding candidates: filter Process Monitor for NAME NOT FOUND on .dll loads
- Linux/macOS equivalents: LD_PRELOAD injection, dylib hijacking via @rpath

Real DLL Hijacks — and Stuxnet's Trust Trick

CS

Researchers have repeatedly reported DLL search-order hijacking in extremely popular tools, including versions of Notepad++ and 7-Zip — a malicious DLL dropped next to the .exe gets loaded silently.

Stuxnet (2010) used stolen, legitimately valid code-signing certificates so Windows would trust and load its malicious drivers without warning.

Section 04

IPC, Traffic Interception & Local Storage

Named pipes, proxying non-HTTP traffic, DPAPI, Registry

What Is IPC? — PrintNightmare, Revisited

CS

IPC (Inter-Process Communication): mechanisms that let separate running programs talk to each other. Common pattern: a normal-user GUI talks to a higher-privileged helper service (SYSTEM/root) to do privileged work.

PrintNightmare was fundamentally an IPC authorization failure — the RPC interface didn't check whether the caller should be allowed to install a driver (which runs as SYSTEM).

Named Pipes

- A named, file-like channel one process creates and others connect to — common for client-service communication
- Check: who is allowed to connect? (ACLs on the pipe)
- Check: does the receiving end authenticate the caller, or just trust whatever arrives?
- Check: does it validate the message content before acting on it?
- Cross-platform equivalents: Unix domain sockets, D-Bus (Linux), XPC (macOS)

Several consumer AV/EDR products have shipped SYSTEM-privileged services exposing named pipes that didn't fully validate messages — letting a low-privileged user trigger privileged file operations through the “trusted” security product itself.

Intercepting Non-HTTP Traffic

- Many desktop apps use custom binary protocols over raw TCP/UDP — the standard web-proxy workflow doesn't directly apply
    1. Identify the protocol/port with Wireshark
    2. Position a generic TCP/UDP relay (Python socket script, or mitmproxy's raw TCP mode) between client and server
    3. Reverse-engineer the message format by comparing captured messages side by side
- If the app does use HTTPS: standard Burp/mitmproxy workflow applies — watch for certificate pinning (bypass via Frida, per Section 3)

Discord Token Theft — Malware You Can Relate To

CS

Discord's Electron client historically stored the user's auth token in a local storage file under %AppData%, in plaintext (or lightly encoded, not tied to re-authentication).

This became one of the most common pieces of “stealer” malware in the wild — simple scripts that read the file and exfiltrate the token, instantly hijacking the account with no password needed.

Signal Desktop — and the Right Way: DPAPI

CS

Researchers (2018) found Signal Desktop stored its SQLite decryption key in plaintext, in a config file next to the encrypted database — because desktop OSes lack a clean equivalent to a phone's secure enclave. A genuine engineering trade-off, not simple incompetence.

DPAPI (Data Protection API) lets Windows apps encrypt data using a key tied to the logged-in user, without managing their own keys — the built-in tool for exactly this problem. Chrome/Edge use it for saved passwords.

Encoding (Base64) ≠ Obfuscation (XOR) ≠ Encryption (AES via DPAPI).

Sample Attack Chain

M1

Architecture

Identify a SYSTEM-privileged helper service — a broken trust boundary by design

→

M2

Static

Review the helper's binary; find the named pipe it listens on

→

M3

Dynamic

Confirm with ProcMon; use Frida to learn the expected message format

→

M4

IPC

Send a crafted message the service doesn't validate → SYSTEM-level write

Recap

01

Architecture

Know the target's type and trust boundaries first — Zoom, LastPass

02

Static Analysis

Read the app without running it — WinRAR's DLL, SolarWinds' password

03

Dynamic Analysis

Watch and manipulate it live — PrintNightmare, real DLL hijacks

04

IPC & Storage

Attack how it talks — PrintNightmare's root cause, Discord, Signal

Toolkit Cheat Sheet

Category	Tools
Static / first contact	file, strings, Ghidra, IDA Free, dnSpy, jadx
Protections check	checksec, winchecksec, PE-bear
Process / file monitoring	Process Monitor, Process Explorer, strace/ltrace
Debuggers	x64dbg, WinDbg, gdb, lldb
Runtime hooking	Frida
Network	Wireshark, Burp Suite, mitmproxy
Windows-specific	Sysinternals suite, regedit

Questions?

Every case study today was public, disclosed, and patched.
Now go & find the next one — legally, and with a scope document.